

Reviewer Pack — Minimal Local Ring Index and Circuit Monotone Width Correlat...

Entry 7092cd304ad9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 16:12:49 UTC

Minimal Local Ring Index and Circuit Monotone Width Correlation

Entry ID: 7092cd304ad9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 16:12:49 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Boolean Circuits

Statement:

For every boolean circuit C with n inputs, the minimal index of its associated local ring $L(C)$ is linearly correlated with its monotone width $w_m(C)$, such that $\text{idx}(L(C)) = \Theta(w_m(C))$. Equivalently: for all circuits C , $\text{idx}(L(C)) \leq \alpha * w_m(C)$, where α is a constant.

Rationale (proposer's reasoning):

Local rings provide a structure to study the complexity of computations, and their minimal index could potentially capture the complexity inherent in boolean circuit computation. The monotone width is a known measure of circuit complexity, and this conjecture suggests that these two structures are related in a quantifiable way, which might expose deeper connections between algebraic and computational complexity.

Taxonomy category: LOCAL_RINGS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c978fd83ee765752

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the correlation coefficient between minimal index $\text{idx}(L(C))$ and monotone width $w_m(C)$ for circuits C with n inputs is ≥ 0.8 and the p-value from a bootstrap test with 30 random seeds is ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Ring Index AND Circuit Monotone Width - Local Ring Theory IN BOOLEAN CIRCUITS - $\text{idx}(L(C)) \leq \alpha * w_m(C)$

Top relevant hits considered: - [<http://arxiv.org/abs/1607.05594v2>] Poincaré series of compressed local Artinian rings with odd top socle degree - [<http://arxiv.org/abs/astro-ph/0405128v1>] Probing the Evolution of the Galaxy Interaction/Merger Rate Using Collisional Ring Galaxies - [<http://arxiv.org/abs/1902.08221v2>] Compact minimal hypersurfaces of index one and the width of real projective spaces - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/hep-th/9707234v2>] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def monotone_width(circuit):
        n = len(circuit)
        if n == 1:
            return 1
        subcircuits = [circuit[i:i+len(circuit)//2] for i in range(0, len(circuit), len(circuit)//2)]
        widths = [monotone_width(sub) for sub in subcircuits]
        return max(widths) + 1

    def local_ring_index(circuit):
        n = len(circuit)
        ring = [[0]*n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if circuit[i] == circuit[j]:
                    ring[i][j] = 1
        generators = []
```

```

    for i in range(n):
        if all(ring[i][j] == ring[j][i] for j in range(n)):
            generators.append(i)
    return len(generators)

def correlation_coefficient(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    var_x = sum((x[i] - mean_x)**2 for i in range(n)) / n
    var_y = sum((y[i] - mean_y)**2 for i in range(n)) / n
    return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

def bootstrap_test(x, y, num_samples=30):
    n = len(x)
    samples = []
    for _ in range(num_samples):
        indices = random.sample(range(n), n)
        x_sample = [x[i] for i in indices]
        y_sample = [y[i] for i in indices]
        samples.append(correlation_coefficient(x_sample, y_sample))
    return sum(samples) / num_samples

n_values = [5, 10, 15, 20, 30, 40]
idxs = []
w_ms = []

for n in n_values:
    for _ in range(5):
        circuit = generate_circuit(n)
        idx = local_ring_index(circuit)
        w_m = monotone_width(circuit)
        idxs.append(idx)
        w_ms.append(w_m)

if len(idxs) < 30 or len(w_ms) < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": len(idxs),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "insufficient_data"
    }

alpha = 0.05
p_value = bootstrap_test(idxs, w_ms)
correlation = correlation_coefficient(idxs, w_ms)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation,
    "instances_tested": len(idxs),
    "n_max": max(n_values),
    "conjecture_holds": correlation >= 0.8 and p_value <= alpha,

```

```

        "counterexample": "" if correlation >= 0.8 and p_value <= alpha else f"correlation={correlation}
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation too low\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its analysis to determine the correlation coefficient and p-value. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	31215
2	preregistration	ollama_remote	glm4:latest	0	0	10507
3	novelty	ollama_remote	glm4:latest	0	0	8605
4	novelty	ollama_remote	glm4:latest	0	0	9795
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18131
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	93505
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51006
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21617
9	judge	ollama_remote	glm4:latest	0	0	50944

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 295325 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7092cd304ad9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7092cd304ad9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7092cd304ad9.tar.gz` (if generated)